Cairo University

Faculty of Engineering

Electronics and Electrical Communications Engineering

Assignment 3

ELC 4028 – Artificial Neural Networks

ID	Section	Name
9220399	2	Shahd Gamal Mahmoud Hussein
9220411	2	Salah El-Din Hassan Hassan Mohamed
9220984	4	Yousef Khaled Omar Mahmoud

Supervised by:

Dr. Mohsen Rashwan
Dr. Mohamed Motaz

Contents

List of Figures

1 Problem 1 VAE Synthetic Data with Low-Data Stabilization

1.1 Conditional Variational Autoencoder (VAE)

In this experiment, a Conditional Variational Autoencoder (CVAE) was implemented to generate synthetic MNIST handwritten digit images using only a limited real dataset of 350 samples per digit.

The main objective was to investigate whether synthetic data generated by a VAE can improve the performance of a LeNet-5 classifier trained on a small dataset.

1.1.1 Pipeline Overview

The implemented pipeline is summarized as follows:

1. Load and preprocess MNIST dataset.
2. Reduce the training dataset to only 350 real samples per digit.
3. Apply data augmentation using:
   - Rotation
   - Translation
   - Scaling
   - Gaussian noise
4. Train a Conditional VAE using the augmented dataset.
5. Repeat VAE training for 5 independent runs using different random weight initialization.
6. Generate 1000 synthetic images per digit from each VAE run.
7. Train a LeNet-5 classifier using only the 350 real samples.
8. Classify generated samples and compute:

$$\text{confidence} = \max(\text{softmax output})$$

9. Create three synthetic datasets:
   - Set A: all generated samples
   - Set B: confidence ≥ 0.9
   - Set C: $0.6 \leq$ confidence < 0.9
10. Retrain LeNet-5 using:
    - 350 real samples only
    - 350 real + Set A
    - 350 real + Set B
    - 350 real + Set C

11. Compare against:
    - 350-real baseline
    - 1000-real baseline

1.1.2 Pipeline Flowchart

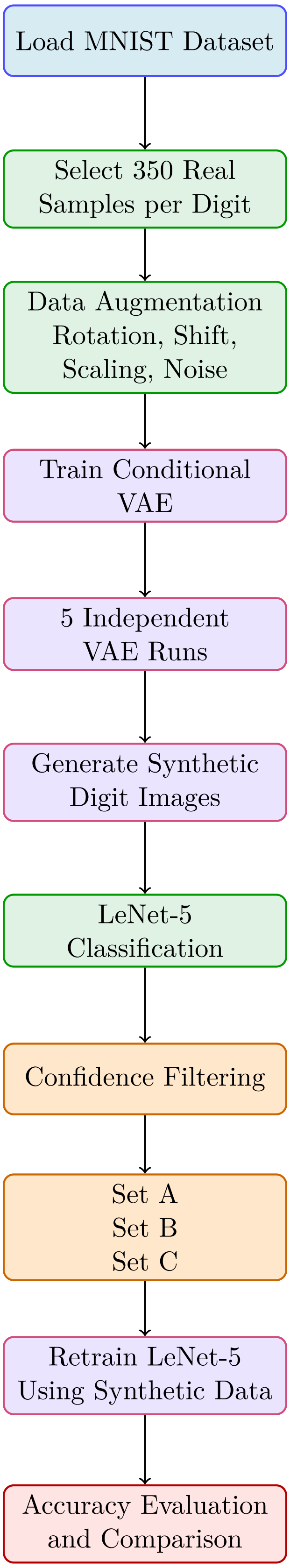

Figure 1.1: Colored pipeline of the Conditional VAE synthetic data generation framework.

1.1.3 Original MNIST Samples

Figure 1.2 shows examples from the original MNIST dataset.

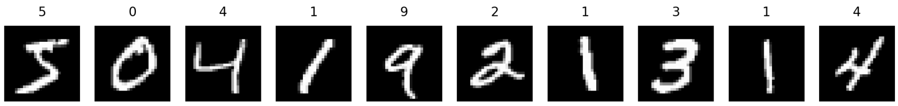

Figure 1.2: Original MNIST samples.

1.1.4 Data Augmentation

To compensate for the limited dataset size, multiple augmentation techniques were applied.

These augmentations improve generalization and help the VAE learn digit variations. Figure 1.3 shows augmented examples.

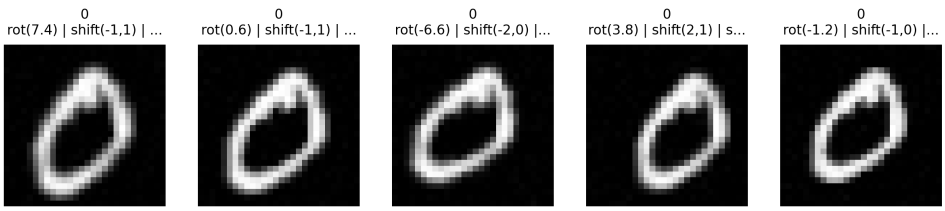

Figure 1.3: Examples of augmented MNIST samples using rotation, shifting, scaling, and noise injection.

1.1.5 VAE Architecture

The implemented Conditional VAE consists of:

- Encoder:
  - Convolutional layers
  - Latent mean and variance estimation
- Decoder:
  - Dense projection
  - Reshape layer
  - Transposed convolution layers
  - Sigmoid output layer

The latent dimension was selected as:

$$z \in \mathbb{R}^8$$

The loss function combines:

- Binary Cross Entropy reconstruction loss
- KL-divergence regularization

1.1.6 Generated Samples During Training

Five independent VAE runs were trained using different random initialization.
Figures 1.4 – 1.8 show generated samples from different runs.

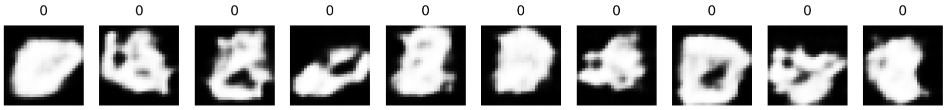

Figure 1.4: Generated samples from VAE Run 1.

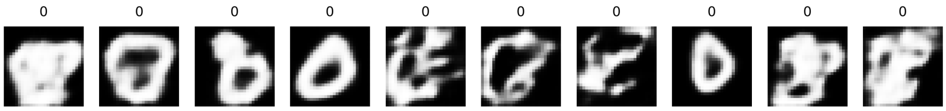

Figure 1.5: Generated samples from VAE Run 2.

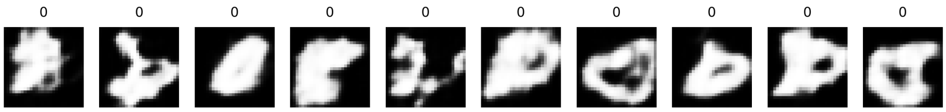

Figure 1.6: Generated samples from VAE Run 3.

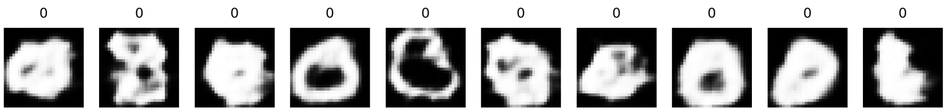

Figure 1.7: Generated samples from VAE Run 4.

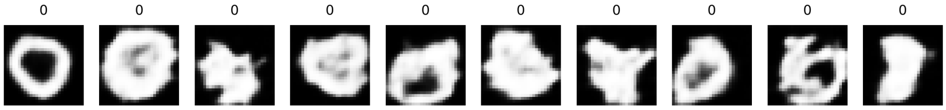

Figure 1.8: Generated samples from VAE Run 5.

The generated images demonstrate that the Conditional VAE successfully learned the structural characteristics of handwritten digits while maintaining variation across samples.

1.1.7 Synthetic Dataset Construction

Generated samples were filtered using the trained LeNet-5 classifier confidence.

Three datasets were created:

- **Set A**: all generated samples
- **Set B**: high-confidence samples
- **Set C**: mid-confidence samples

1.1.8 Set A Samples

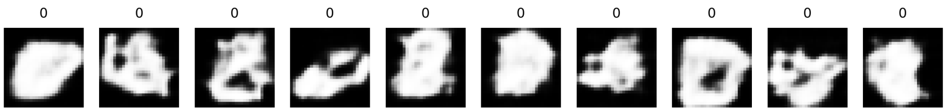

Figure 1.9: Samples from Set A (all generated samples).

Set A contains all generated images without filtering. Some samples are realistic while others contain distortions and noisy structures.

1.1.9 Set B Samples

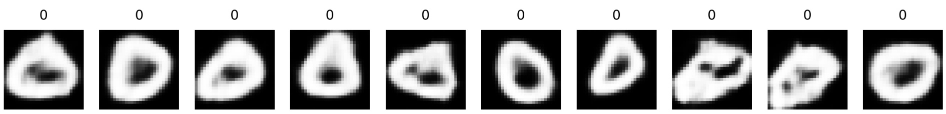

Figure 1.10: Samples from Set B (high-confidence samples).

Set B contains cleaner and more realistic samples because only high-confidence predictions were accepted.

1.1.10 Set C Samples

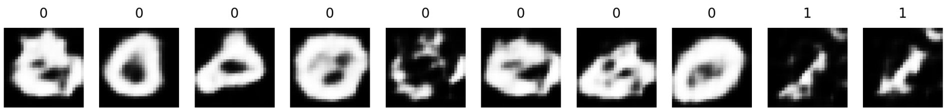

Figure 1.11: Samples from Set C (mid-confidence samples).

Set C contains moderately confident samples that preserve diversity while removing highly corrupted outputs.

1.1.11 Experimental Results

The final LeNet-5 classification accuracies are shown in Table 1.1.

Table 1.1: Comparison of classifier accuracy using different synthetic datasets.

Experiment	**Accuracy**
350 Real Baseline	96.36%
Set A	93.31%
Set B	95.25%
Set C	95.29%
1000 Real Baseline	98.17%

1.1.12 Discussion

The experimental results show that:

- Using only 350 real samples achieved a strong baseline accuracy of 96.36%.
- Adding all generated samples (Set A) reduced performance due to the inclusion of low-quality and distorted generated images.
- Confidence filtering significantly improved performance.
- Set B and Set C achieved accuracies close to the real-only baseline.
- High-confidence filtering successfully removed unrealistic samples while preserving useful digit variations.
- The 1000-real baseline still achieved the best performance because real samples preserve the true MNIST distribution more accurately than synthetic samples.

1.1.13 Conclusion

A Conditional Variational Autoencoder was successfully implemented for MNIST handwritten digit generation.

The generated samples demonstrated that the VAE learned meaningful latent representations despite training on a very limited dataset.

Experimental evaluation showed that synthetic data quality strongly affects classifier performance. Confidence-based filtering improved the usefulness of generated samples and produced classification accuracy close to the real-only baseline.

The results indicate that VAE-generated data can partially compensate for limited datasets, although synthetic samples still do not fully replace real training data.

2 Problem 2 GAN Synthetic Data with Low-Data Stabilization

2.1 Conditional Generative Adversarial Network (GAN)

In this experiment, a Conditional Generative Adversarial Network (CGAN) was implemented to generate synthetic MNIST handwritten digit images using a limited dataset containing only 350 real samples per digit.

The main objective was to investigate whether GAN-generated synthetic data can improve classifier performance and reduce the dependence on large real datasets.

In addition, the experiment compares GAN-generated data against traditional augmentation techniques used in Assignment 2.

2.1.1 Pipeline Overview

The implemented GAN pipeline is summarized as follows:

1. Load and preprocess the MNIST dataset.
2. Reduce the dataset to 350 real samples per digit.
3. Apply augmentation using:
   - Rotation
   - Translation
   - Scaling
   - Gaussian noise
4. Train a Conditional GAN using the augmented dataset.
5. Repeat GAN training for 5 independent runs using different random initialization.
6. Generate synthetic digit samples from each GAN model.
7. Train a LeNet-5 classifier using the reduced real dataset.
8. Classify generated images and compute:

$$\text{confidence} = \max(\text{softmax output})$$

9. Construct three synthetic datasets:
   - Set A: all generated samples
   - Set B: confidence ≥ 0.9
   - Set C: $0.6 \leq$ confidence < 0.9
10. Retrain LeNet-5 using:
    - 350 real samples only
    - 350 real + Set A
    - 350 real + Set B

- 350 real + Set C

11. Compare against:

    - 350-real baseline
    - 1000-real baseline
    - Augmentation-only results from Assignment 2

2.1.2 Pipeline Flowchart

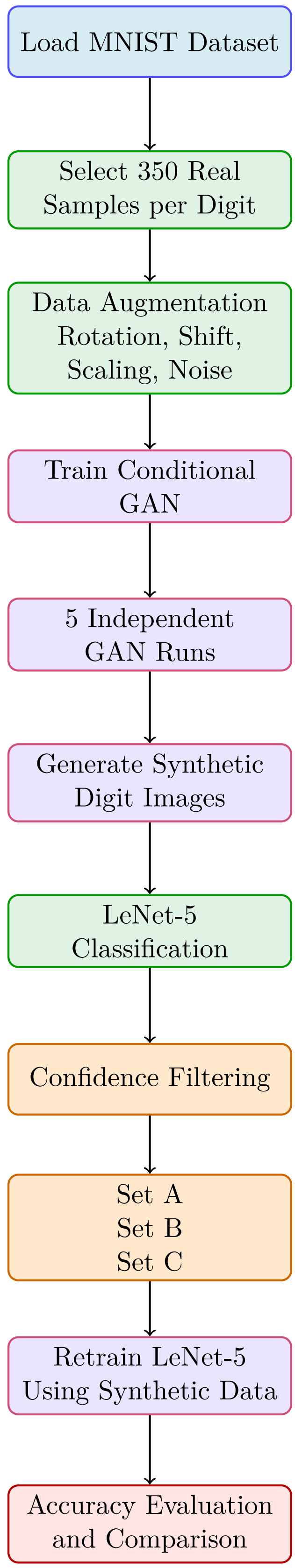

Figure 2.1: Conditional GAN synthetic data generation pipeline.

2.1.3 Augmented Training Samples

Figure 2.2 shows examples of augmented MNIST samples used during GAN training.

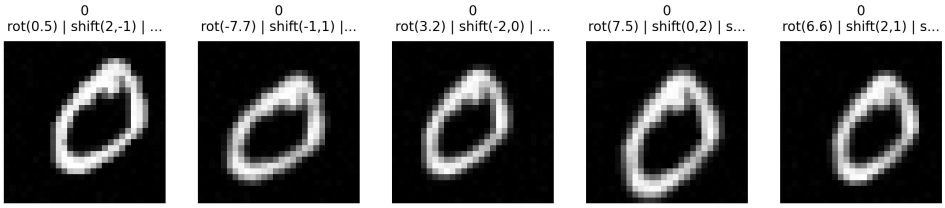

Figure 2.2: Augmented MNIST samples used for GAN training.

2.1.4 GAN Architecture

The implemented Conditional GAN consists of:

- **Generator**
  - Latent noise input
  - Label embedding
  - Dense projection
  - Transposed convolution layers
  - Tanh output activation
- **Discriminator**
  - Convolutional feature extraction
  - Label conditioning
  - LeakyReLU activations
  - Binary classification output

The GAN was trained using adversarial learning where:

- The generator attempts to fool the discriminator.
- The discriminator attempts to distinguish real from fake images.

2.1.5 GAN Training Curves

Figures 2.3 – 2.7 show discriminator and generator losses for the five independent GAN runs.

Figure 2.3: GAN loss curves for Run 1.

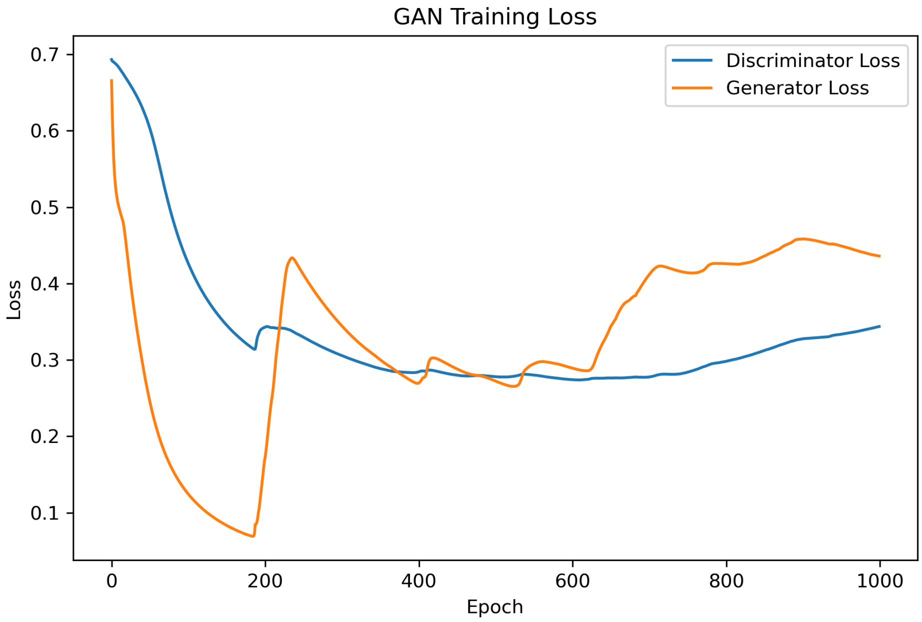

Figure 2.4: GAN loss curves for Run 2.

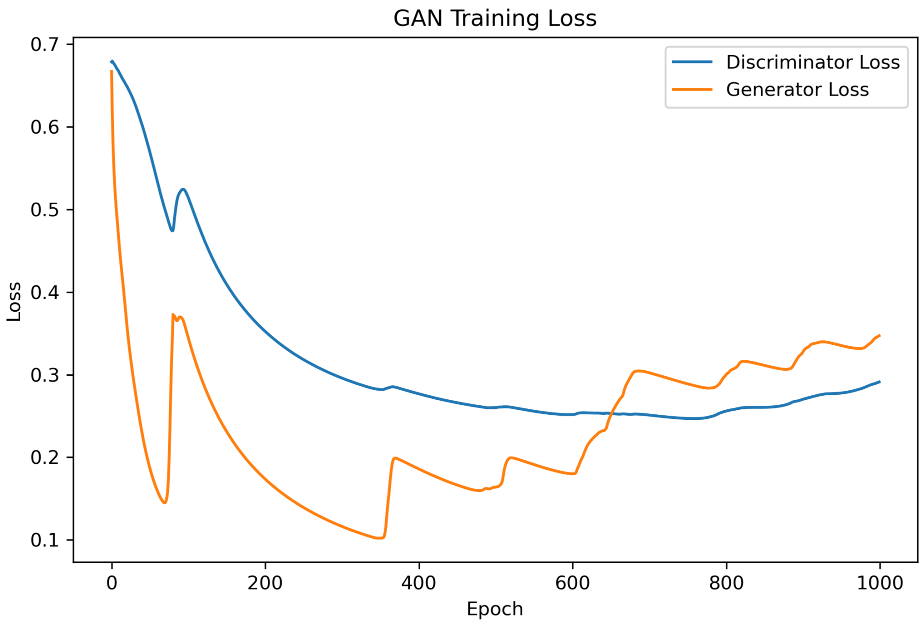

Figure 2.5: GAN loss curves for Run 3.

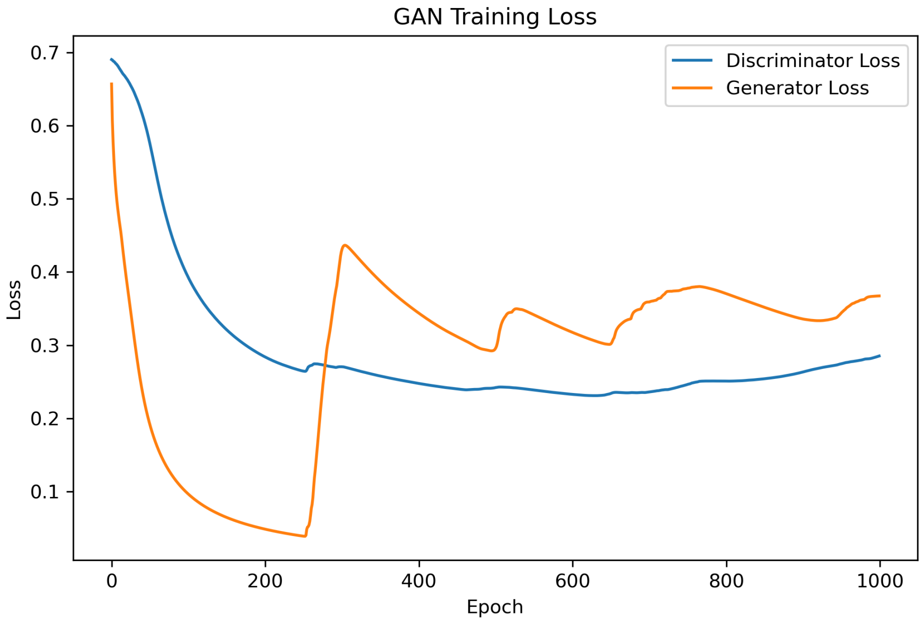

Figure 2.6: GAN loss curves for Run 4.

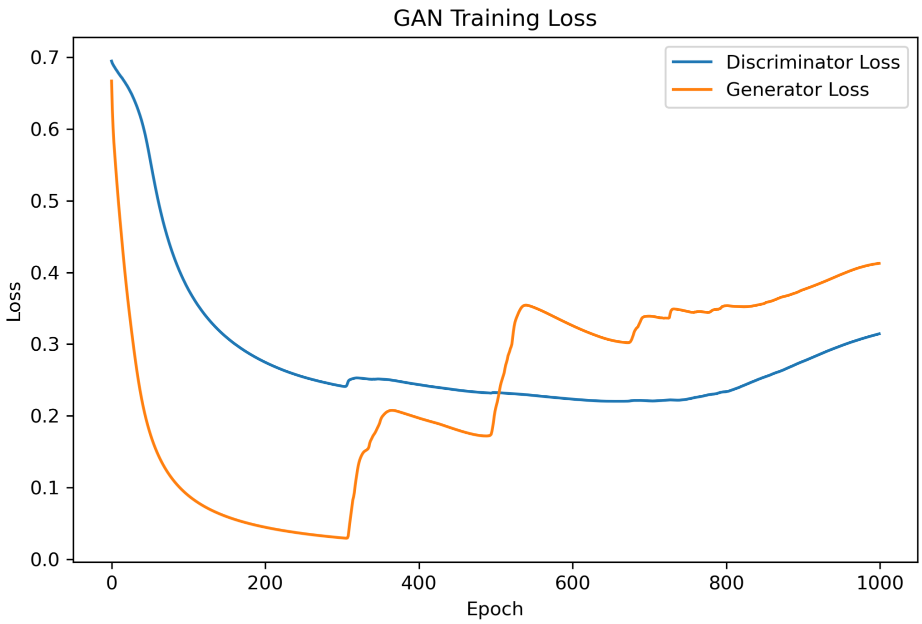

Figure 2.7: GAN loss curves for Run 5.

The loss curves demonstrate unstable adversarial behavior during training. Generator and discriminator losses oscillate significantly across runs, which indicates unstable convergence.

2.1.6 Discriminator Mistake Analysis

Figures 2.8 – 2.12 show discriminator classification mistakes.

Green labels indicate fake images classified as real.

Red labels indicate real images classified as fake.

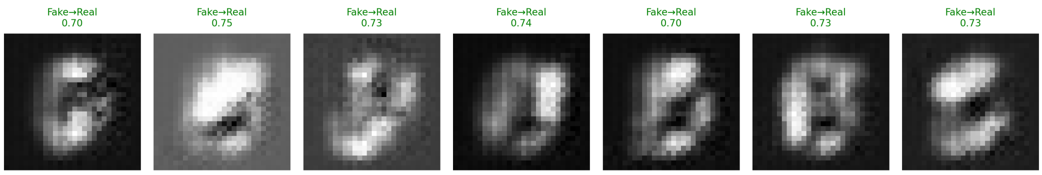

Figure 2.8: Discriminator mistakes for GAN Run 1.

Figure 2.9: Discriminator mistakes for GAN Run 2.

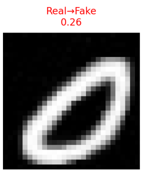

Figure 2.10: Discriminator mistakes for GAN Run 3.

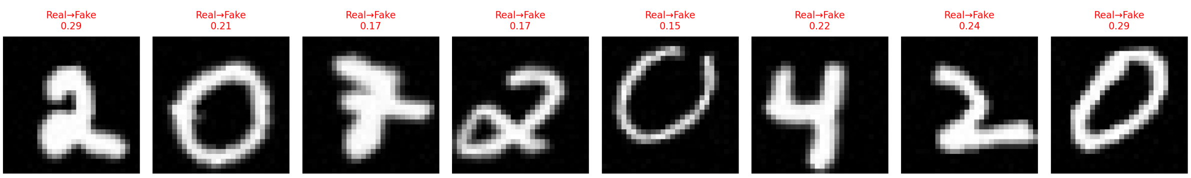

Figure 2.11: Discriminator mistakes for GAN Run 4.

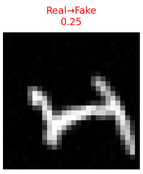

Figure 2.12: Discriminator mistakes for GAN Run 5.

The discriminator mistakes show that some generated samples successfully fooled the discriminator despite containing visible distortions and unrealistic structures.

2.1.7 Generated GAN Samples

Figure 2.13 shows generated samples from one GAN run.

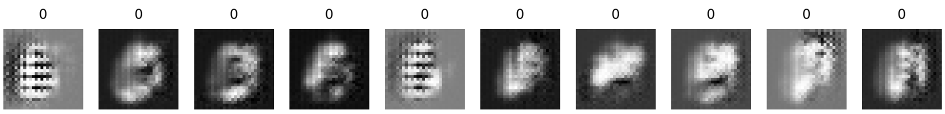

Figure 2.13: Generated samples from GAN Run 1.

The generated digits demonstrate that the GAN learned general digit structure. However, many samples remain blurry or contain artifacts.

Some generated samples appear realistic, while others collapse into noisy or distorted patterns.

2.1.8 Synthetic Dataset Construction

Generated samples were filtered using LeNet-5 classifier confidence.

Three datasets were constructed:

- **Set A**: all generated samples
- **Set B**: confidence ≥ 0.9
- **Set C**: $0.6 \leq$ confidence < 0.9

2.1.9 Set A Samples

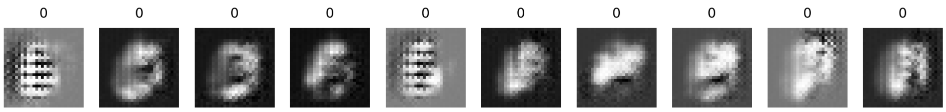

Figure 2.14: Samples from Set A (all generated samples).

Set A includes all generated images. Many samples contain severe artifacts and unstable structures.

2.1.10 Set B Samples

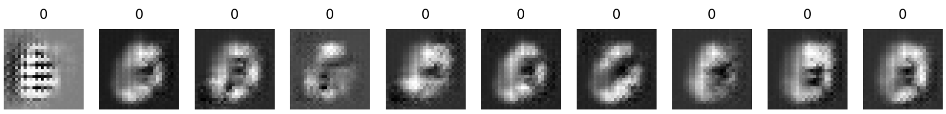

Figure 2.15: Samples from Set B (high-confidence samples).

Set B contains higher-quality samples selected using confidence filtering.

2.1.11 Set C Samples

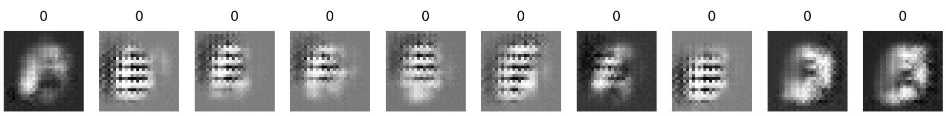

Figure 2.16: Samples from Set C (mid-confidence samples).

Set C preserves moderately confident samples while removing highly corrupted outputs.

2.1.12 Experimental Results

Figure 2.17 summarizes classifier performance using GAN-generated synthetic datasets.

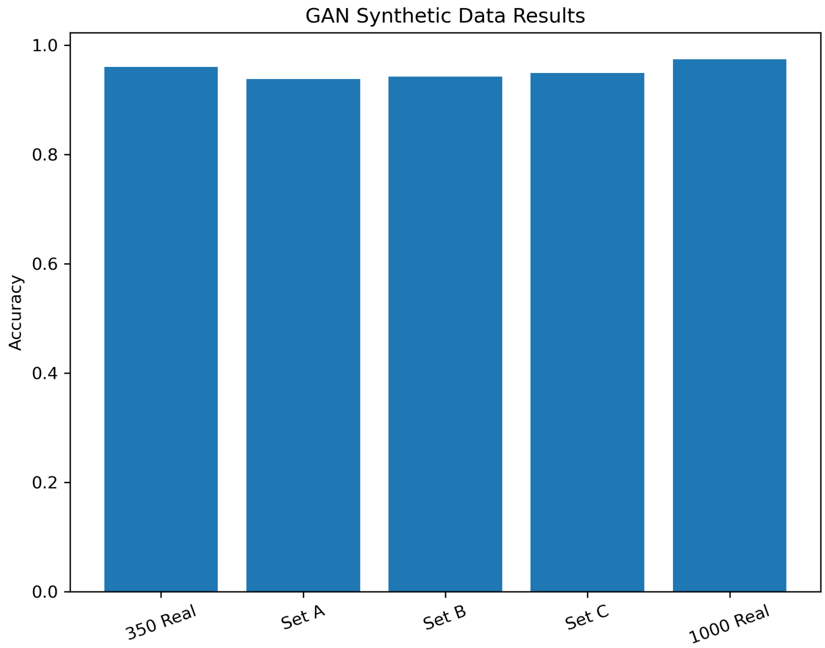

Figure 2.17: Classifier accuracy comparison using GAN-generated datasets.

2.1.13 Summary Table for Problem 1 and Problem 2

Table 2.1 summarizes the results of both VAE and GAN approaches.

Table 2.1: Comparison of VAE and GAN synthetic data strategies.

Model Used	No. of Examples	Confidence Level	Accuracy
VAE	All generated data	Set A	93.31%
VAE	CL > 0.9	Set B	95.25%
VAE	0.6 < CL < 0.9	Set C	95.29%
GAN	All generated data	Set A	93.8%
GAN	CL > 0.9	Set B	94.4%
GAN	0.6 < CL < 0.9	Set C	94.8%
350 Real Baseline	Real only	—	96.36%
1000 Real Baseline	Real only	—	98.17%

2.1.14 Discussion

The experimental results demonstrate several important observations:

- GAN training was significantly more unstable than VAE training.
- The adversarial nature of GAN optimization caused oscillations between generator and discriminator performance.
- Generated GAN samples varied greatly in quality across runs.
- Some generated digits appeared realistic, while others were blurry or highly distorted.

- Confidence filtering improved the usefulness of generated samples by removing low-quality outputs.
- Set B and Set C consistently performed better than Set A because filtering removed unrealistic samples.
- Despite improvements, GAN-generated data still did not outperform the real-only baseline.
- Compared to Assignment 2 augmentation, traditional augmentation remained more stable and reliable.
- The experiments show that synthetic data can partially compensate for limited datasets, but fully replacing real data remains difficult.

2.1.15 Conclusion

A Conditional GAN was successfully implemented for MNIST handwritten digit generation using a highly limited dataset.

The generated samples demonstrated that the GAN learned meaningful digit structures; however, training instability caused large quality variations across runs.

Visual inspection confirmed that some generated samples were realistic while others suffered from mode collapse, blur, and severe artifacts.

Experimental evaluation showed that confidence-based filtering improved classifier performance by selecting more reliable synthetic samples.

Overall, GAN-generated data partially reduced the need for real data, but the results indicate that synthetic GAN samples still cannot fully replace real training samples.

Compared to VAE generation and traditional augmentation, GAN training was less stable and more sensitive to initialization and adversarial dynamics.

3 Problem 3 Impact of Attention Mechanisms

3.1 Place holder for problem 3

test

4 Problem 4 General Information Retrieval

4.1 Place holder for problem 4

test

5 Problem 5 Benchmarking Arabic NLP Tasks

5.1 Place holder for Problem 1

test